Task Management Flask API

Deployed on AWS Elastic Beanstalk

Project Summary

A simple Flask-based Task Management API with a SQLite database backend and a lightweight HTML/JavaScript frontend, built and deployed to AWS Elastic Beanstalk in the ap-south-1 (Mumbai) region.

Tech Stack

• Backend: Python, Flask, Flask-SQLAlchemy
• Database: SQLite
• Web server: Gunicorn
• Frontend: HTML, CSS, vanilla JavaScript (fetch API)
• Hosting: AWS Elastic Beanstalk (Python 3.12/3.13 platform, Amazon Linux 2023)
• Region: Asia Pacific (Mumbai) - ap-south-1

Project Structure

```
task-management/
■■■ app.py              # Flask application (routes + models)
■■■ requirements.txt    # Python dependencies
■■■ Procfile             # gunicorn start command
■■■ make_zip.py          # Packaging script for deployment
■■■ templates/
■   ■■■ index.html       # Frontend UI
■■■ venv/                # Local virtual environment (not deployed)
```

requirements.txt

```
flask==3.0.3
flask-sqlalchemy==3.1.1
gunicorn==22.0.0
```

Procfile

```
web: gunicorn app:app
```

app.py

```
import os
from flask import Flask, jsonify, request, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
basedir = os.path.abspath(os.path.dirname(__file__))
app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:///{os.path.join(basedir, 'tasks.db')}"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db = SQLAlchemy(app)


class Task(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    done = db.Column(db.Boolean, default=False)

    def to_dict(self):
        return {"id": self.id, "title": self.title, "done": self.done}


with app.app_context():
    db.create_all()


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/tasks", methods=["GET"])
def get_tasks():
    tasks = Task.query.all()
    return jsonify([t.to_dict() for t in tasks])


@app.route("/tasks", methods=["POST"])
def create_task():
    data = request.get_json()
    if not data or "title" not in data:
        return jsonify({"error": "title is required"}), 400
    task = Task(title=data["title"])
    db.session.add(task)
    db.session.commit()
    return jsonify(task.to_dict()), 201


@app.route("/tasks/<int:task_id>", methods=["PUT"])
def update_task(task_id):
    task = Task.query.get_or_404(task_id)
    data = request.get_json()
    task.title = data.get("title", task.title)
    task.done = data.get("done", task.done)
    db.session.commit()
    return jsonify(task.to_dict())


@app.route("/tasks/<int:task_id>", methods=["DELETE"])
def delete_task(task_id):
```

```
    task = Task.query.get_or_404(task_id)
    db.session.delete(task)
    db.session.commit()
    return "", 204


if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

Deployment Steps (AWS Console)

1. Install the Flask app locally and confirm it runs with `python app.py`.
2. Add gunicorn to requirements.txt (required for production serving on Elastic Beanstalk).
3. Package the project into a zip containing app.py, requirements.txt, Procfile, and the templates/ folder — using a Python zipfile script to avoid Windows backslash path issues.
4. Go to AWS Console → Elastic Beanstalk → Create application.
5. Choose Platform: Python, branch Python 3.12/3.13, recommended version.
6. Choose Application code → Local file → upload the deployment zip.
7. Choose the Single instance (free tier eligible) preset.
8. Accept defaults for service role, EC2 instance profile, networking, and database, then Submit.
9. Wait for the environment health to turn green, then open the environment URL to test.
10. For future updates: Upload and deploy → choose the new zip → set a new version label → Deploy.

Issues Encountered and Fixes

TemplateNotFound: index.html

The templates/ folder was missing from the deployment zip. Fixed by writing a make_zip.py script that explicitly walks the templates folder and adds each file with forward-slash paths.

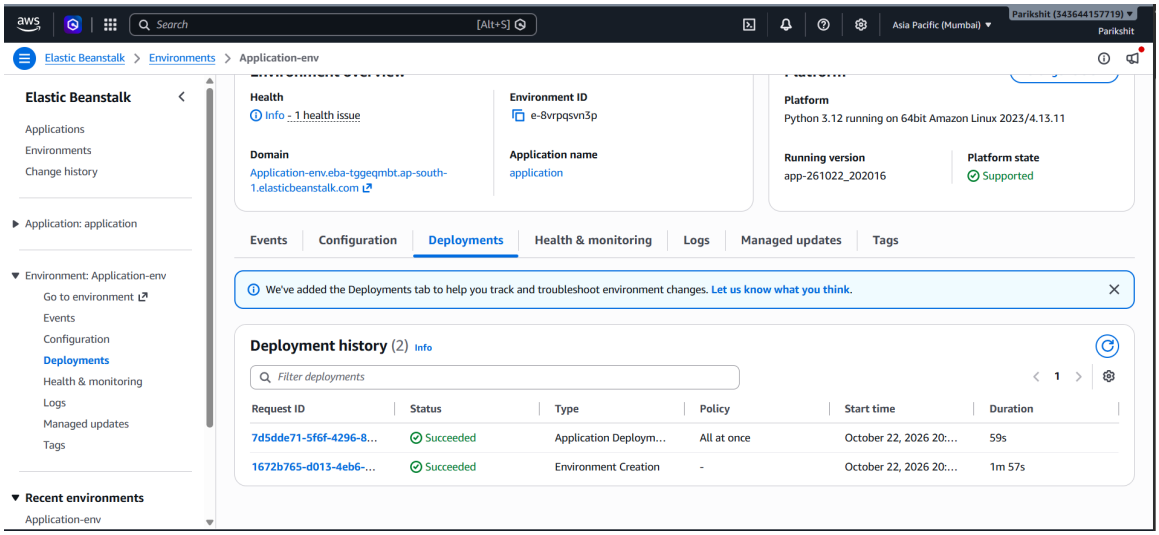
Backslashes as path separators

Zips created with Windows tools (Compress-Archive) sometimes used backslash path separators, which Linux unzip on the EB instance rejected. Fixed by building the zip with Python's zipfile module and replacing os.sep with '/' in archive names.

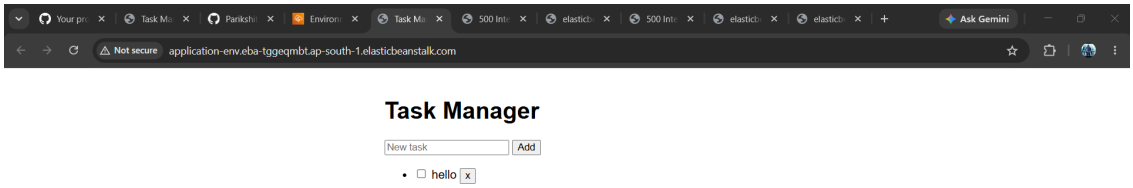
Internal Server Error after deploy

Caused by the same TemplateNotFound issue showing up in production logs (web.stdout.log), confirmed via the Elastic Beanstalk Logs tab.

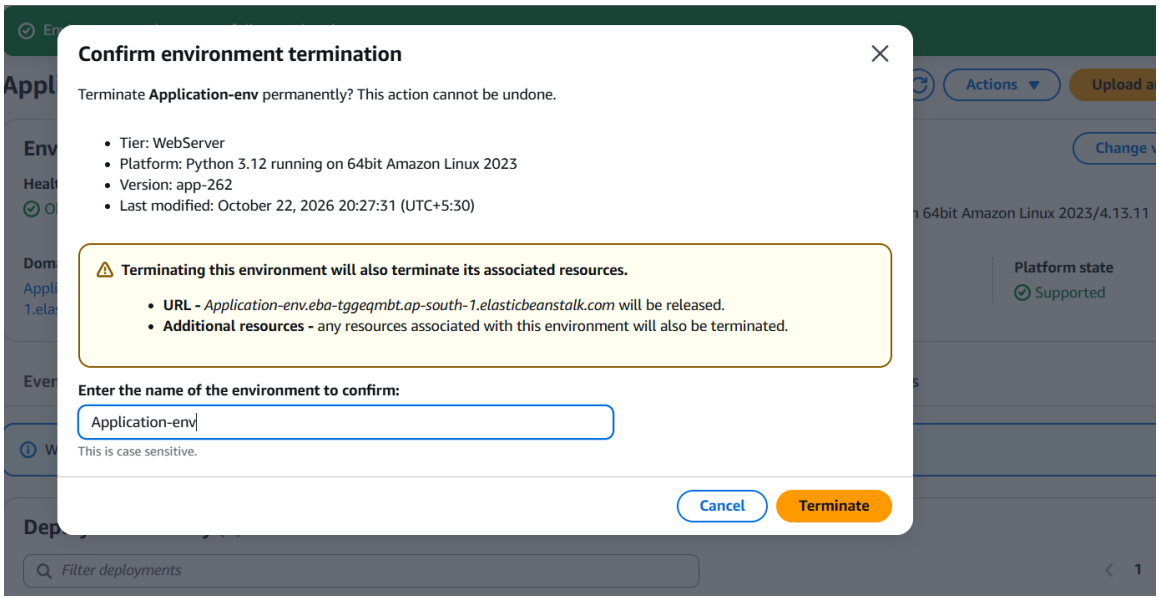
Screenshots



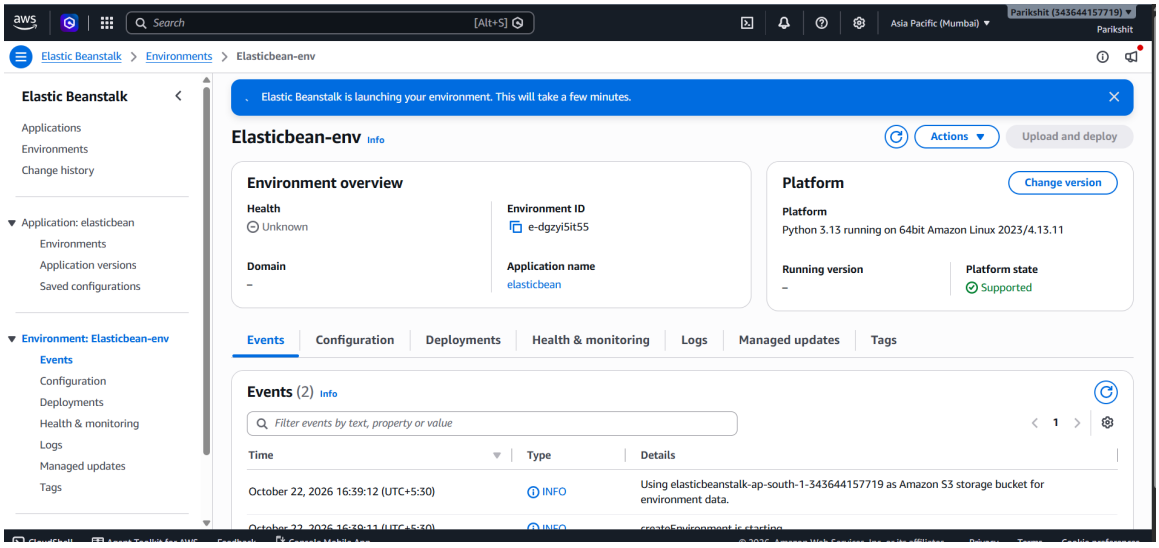
Elastic Beanstalk environment overview showing health, platform, and successful deployment history.



Deployed Task Manager app running live on the Elastic Beanstalk URL.



Environment termination confirmation dialog, used to clean up resources after testing.



Creating a new Elastic Beanstalk environment via the AWS Console, uploading the local deployment zip.

Result

The application was successfully deployed and verified working: tasks can be added, marked complete, and deleted through the live Elastic Beanstalk URL, backed by a persistent SQLite database on the instance.